Improved Upper Bounds on the Size of Condorcet Domains

Pavlos Moros Søren Riis

Review draft with examples, 1 November 2026

Abstract

We prove $f(n) \le 32nf(n-1)/(11n+8)$ for the maximum size of a Condorcet domain, giving exponential base $32/11 < 3$. An exact deletion count, a decomposition into adjacent-swap components and a local counting inequality give the bound; the same local inequality gives base $8/3$ for peak-pit domains. Small examples illustrate the structural counts. All proofs and the complete finite computation, including its exact data and coverage argument, are included in the paper and appendices.

1 Introduction

A Condorcet domain admits rankings for which pairwise majority is transitive for every odd number of voters, allowing repeated rankings. How many rankings can be admitted while retaining consistent majority decisions? Write $f(n)$ for the maximum on n alternatives. We use Sen's equivalent combinatorial description: each triple has an alternative forbidden from one of its three positions in every admitted ranking [1]. A domain is *peak-pit* when these forbidden positions can always be chosen first or last. The alternative and the position may vary with the triple; no common axis is required. Write $h(n)$ for the peak-pit maximum.

Fishburn [2] surveys the extremal problem, and Raz [3] proved an exponential upper bound for $f(n)$. For peak-pit constructions and recent lower bounds, see Karpov and Slinko [4] and Karpov, Markström, Riis and Zhou [5]. An earlier AI-assisted working draft of this project [6] recorded the exact fibre count, the gap inequality, and bounds with general bases $56/19$ and $1336/455$, and peak-pit base $8/3$. Its arguments and verification programs were developed with Claude and examined with Codex; its finite correction coefficients were selected by linear programming. The present paper consolidates that investigation, adds the component reduction, and improves the general base to $32/11 = 2.90909\ldots$.

Theorem 1.1 (Main bounds)**.** *For every integer $n \ge 3$,*

$$f(n) \le \frac{32n}{11n+8} f(n-1), \qquad h(n) \le \frac{8}{3} h(n-1).$$

Consequently $f(n) \le 4(32/11)^{n-3}$, $h(n) \le 4(8/3)^{n-3}$, and $f(n) = O(n^{-8/11}(32/11)^n)$. The limits $\lim_{n\to\infty} f(n)^{1/n}$ and $\lim_{n\to\infty} h(n)^{1/n}$ exist and are at most $32/11$ and $8/3$, respectively.

The structural step partitions a domain, after completion if needed, into connected components under swaps of consecutive entries. Each component is described by never-first/never-last restrictions and fixed comparisons; its deletion outputs count each consecutive range of insertion positions separately. The proof uses majority projection, related to the median structure studied by Puppe and Slinko [7], and proves the required component statement directly. A deletion estimate on these components then combines with the gap inequality to give the general bound. The only computational ingredient is a local inequality; Appendices B–C include every datum and prove that its checking procedure is exhaustive.

2 Orders, triple restrictions and full families

Let X be a finite set of n alternatives. An *order* is a strict total order on X, written from first to last; $a > b$ means that a occurs before b. Restricting an order to a subset means erasing all other labels; this restriction is also called its *trace* on the subset. For a triple $T \subseteq X$ and $x \in T$, the *never-condition* $x\mathrm{N}_i$, for $i \in \{1, 2, 3\}$, says that x does not occupy position i in the restriction to T. Thus $x\mathrm{N}_1$, $x\mathrm{N}_2$ and $x\mathrm{N}_3$ mean never first, never middle and never last, respectively, *within that triple*. For instance, $2\mathrm{N}_3$ on $\{1, 2, 3\}$ permits exactly $123, 213, 231, 321$. A condition is *common* to a family if every order satisfies it. We also call a never-condition a *law*.

Definition 2.1 (Condorcet and peak-pit domains)**.** A *Condorcet domain* is a family of orders with a common never-condition on every triple. It is *peak-pit* if each triple has a common never-first or never-last condition. We call these two kinds of condition *extreme*; never-middle is the remaining kind.

By Sen's characterisation [1], this is equivalent to the voting description in the introduction. Appendix A.1 includes an elementary proof. The functions $f(n), h(n)$ are attained maxima because the set of orders is finite. A Condorcet domain is *maximal* if no further order can be added while preserving the Condorcet property; a *maximum* domain has largest possible cardinality. The first term concerns adding orders; the second concerns the largest size among all domains on the given alternatives.

Definition 2.2 (Full families and $\mathcal{F}_n$)**.** Choose one never-condition for each triple, and collect *every* order satisfying all the chosen conditions. We call the resulting family a *full domain* for that choice of conditions. To define $\mathcal{F}_n$, use the labels $0, \ldots, n-1$, choose an extreme condition on every triple, and optionally require specified pair comparisons, such as $a > b$. The nonempty families consisting of *all* orders obeying such a system form $\mathcal{F}_n$.

Here "full" refers to satisfying a chosen system exactly; it does not mean maximal. It also differs from the literature's "closed", meaning closed under majority [7]. Families on other label sets are tested for membership in $\mathcal{F}_n$ after relabelling by a bijection; all conditions and counts are invariant under this change of names.

Every maximal Condorcet domain is full: choose a common condition on each triple. All orders obeying the choices form a Condorcet enlargement, so maximality forces equality. A maximum peak-pit domain likewise equals its extreme-condition completion and belongs to $\mathcal{F}_n$. This completion need not be maximal among all Condorcet domains. Whenever we delete a label we assume $n \geq 1$.

For any family D, the *swap graph* has its orders as vertices; two are joined when exchanging consecutive entries takes one to the other. Such a swap is *enabled*. A path is a sequence of these swaps staying in D. A *swap component* consists of all orders reachable from a given order, including itself, by such paths.

3 Insertions and the exact count

For a family D on X, with $|X| = n$, define

$$D_{-x} = \{\sigma \text{ with } x \text{ erased} : \sigma \in D\}, \qquad \Sigma(D) = \sum_{x \in X} |D_{-x}|.$$

Repeated outputs are counted only once in each D_{-x}. If D is a Condorcet domain, restriction preserves its triple laws, so

$$\Sigma(D) \le nf(n-1), \qquad \Sigma(D) \le nh(n-1) \quad \text{if } D \text{ is peak-pit.} \tag{1}$$

For $P \in D_{-x}$, write $P[x : t]$ for inserting x with t labels above it; t indexes its insertion slot. Its *insertion fibre* is

$$F_D(x, P) = \{t \in \{0, \dots, n-1\} : P[x : t] \in D\}.$$

A *run* is a maximal consecutive block of occupied slots. A fibre with r runs contributes $r - 1$ to the total gap count $G(D)$; a long empty stretch between runs is one gap. Let $A(D)$ count pairs (σ, j), where $\sigma \in D$ and the swap at positions $j, j+1$, $1 \le j < n$, is enabled. Thus each edge counts twice. For a fixed family D, we abbreviate these statistics to $M = |D|,\ \Sigma = \Sigma(D),\ G = G(D),\ A = A(D)$.

The starting point is an exact count, with no inequality or lost term: every order/label choice is accounted for by a deletion output, a gap or an enabled-swap incidence.

Lemma 3.1 (Fibre identity)**.** *For every family of orders,* $\boxed{\Sigma + G + A = nM}$.

Proof. A fibre with m occupied slots and r runs has $m - r$ neighbouring occupied pairs. Summing m gives nM, and summing r gives $\Sigma + G$. Each neighbouring occupied pair is a swap edge with the inserted label distinguished. Every edge occurs twice, once for each exchanged label, giving exactly A. □

Example 3.2 (Deletion collisions)**.** The full domain $D = \{123, 213, 231, 321\}$ satisfies $2\mathrm{N}_3$. Its deletion outputs are

$$D_{-1} = \{23, 32\}, \qquad D_{-2} = \{13, 31\}, \qquad D_{-3} = \{12, 21\}.$$

For instance, deleting 1 merges $123, 213, 231$ into 23, and $F_D(1, 23) = \{0, 1, 2\}$. The swap graph is the path $123 \leftrightarrow 213 \leftrightarrow 231 \leftrightarrow 321$. Thus $M = 4, \Sigma = 6, G = 0, A = 6$, and $6 + 0 + 6 = 3 \cdot 4$.

The *top slice at x* retains only orders starting with x and then erases x. The *bottom slice at x* does the same for orders ending with x. A slice can be empty. A sum over slices runs over the choices of fixed endpoint label.

Lemma 3.3 (Two closure properties)**.** *Families in $\mathcal{F}_n$ have interval insertion fibres (one run). Every nonempty slice obtained by fixing and removing a first or last label belongs to $\mathcal{F}_{n-1}$.*

Proof. Fix the relative order of the other labels. An extreme law permits an interval of slots for the inserted label: never-first puts it below the higher of the two others, and never-last above the lower. If another label is distinguished, the law is automatic or puts the inserted label on one specified side of it. Fixed comparisons also give intervals. Intersect these intervals and use fullness. For an endpoint slice, internal triple laws remain. A triple containing the removed endpoint reduces to a fixed comparison, a tautology or a contradiction. Pair constraints reduce similarly; contradictions mean the slice is empty. The remaining constraints describe the slice exactly. □

4 Reducing a general domain to extreme laws

Throughout this section D is full. Its swap components are the connected pieces of the graph defined in Section 2. For three orders write $\mathrm{maj}(\alpha, \beta, \gamma)$ for their pairwise majority order. The majority-closure argument is part of the median structure studied by Puppe and Slinko [7]; we give the specific adjacent-swap component argument in full.

Lemma 4.1 (Majority projection)**.** *The majority of three members of D lies in D. If K is a swap component of D and U is the set of all oriented pair comparisons common to every order of K, then*

$$K = \{\tau \in D : \tau \text{ satisfies every comparison in } U\}.$$

Proof. Majority is an order by Appendix A.1. On a triple $\{x, y, z\}$, the law $x\mathrm{N}_2$ says $x > y$ if and only if $x > z$; the two majority comparisons therefore agree as well. If majority violated a never-first law, two sets of at least two voters among three would support its two comparisons. Their intersection would contain a voter violating the law. Never-last is symmetric. Fullness therefore puts the majority order in D.

For the displayed equality, suppose $\tau \in D$ satisfies U. Choose $\sigma \in K$ disagreeing with τ on as few pair comparisons as possible. If they differ, a differing comparison is not fixed on K. Take $\rho \in K$ agreeing with τ there, and project a path from σ to ρ by $v \mapsto \mathrm{maj}(\sigma, \tau, v)$. Images lie in D; consecutive images differ in at most one comparison. Two distinct orders differing in one comparison exchange adjacent labels: an intervening label would force another comparison to change. Remove consecutive repeated images; the remaining path stays in K and starts at σ. On comparisons where σ, τ agree it is constant; elsewhere it follows v. Its last order corrects the selected disagreement without losing any agreement. This contradicts the choice of σ, proving the claim. □

Proposition 4.2 (Component identity)**.** *Every component K belongs to $\mathcal{F}_n$, and*

$$\sum_K |K| = M, \qquad \sum_K \Sigma(K) = \Sigma(D) + G(D). \tag{2}$$

Proof. A chosen never-middle law $x\mathrm{N}_2$ equates the two comparisons of x on that triple. Neither can change along a swap path, which changes only one comparison at a time. Thus throughout K, x is above both others or below both. Replace the law by $x\mathrm{N}_3$ or $x\mathrm{N}_1$, respectively. Keep the original extreme laws and add all comparisons U fixed on K. Every order in K satisfies these constraints. Conversely, they recover the old never-middle laws, so an order satisfying them lies in D and agrees with U. Lemma 4.1 puts it in K. Thus these conditions describe K exactly, proving $K \in \mathcal{F}_n$.

Components partition D, giving the first identity. Each original fibre run lies in one component, because consecutive occupied slots are swap edges. A component cannot contain two separated runs: its own fibre is an interval by Lemma 3.3. Thus each original run gives exactly one component deletion output. Summing runs gives the second identity. □

Example 4.3 (Counting separated runs)**.** The full domain $D = \{123, 132, 231, 321\}$, defined by $1\mathrm{N}_2$, has components $\{123, 132\}$ and $\{231, 321\}$. Its 1-fibres above 23 and 32 both have slots $\{0, 2\}$, each contributing one gap. Thus $\Sigma(D) = 6, G(D) = 2$. Each component has deletion sum 4, so $4 + 4 = 6 + 2$: splitting the family counts its separated insertion runs separately.

5 The gap inequality

Fix a Condorcet domain D on X, with $|X| = n$. A *lift* moves one label upwards, and a *drop* moves it downwards, keeping all other relative positions and remaining in D. These moves may jump several slots. An incidence (σ, x) is *free* if it has both moves, and *frozen* if it has neither; their counts are $\mathrm{fr}(D), \mathrm{fz}(D)$. In an order σ, consecutive entries a, c form a *locked* pair if a has no drop and c has no lift. Let $\mathrm{lk}(D)$ count these triples (σ, a, c). With D fixed, abbreviate these counts to $\mathrm{fr}, \mathrm{fz}, \mathrm{lk}$.

Lemma 5.1 (Two counts)**.** *For a full domain,*

$$2\Sigma = nM - \mathrm{fr} + \mathrm{fz} = (n+1)M - A + \mathrm{lk}, \qquad 3\Sigma + \mathrm{fr} = (n+1)M + G + \mathrm{fz} + \mathrm{lk}\,. \tag{3}$$

Proof. Each fibre has one order with no lift and one with no drop. Their combined count is 2Σ; an incidence contributes 0, 1 or 2 according as it is free, has one move, or is frozen. For an adjacent pair, the missing drop of its upper label and missing lift of its lower label contribute 0 if the swap is enabled, 2 if locked, and 1 otherwise. Indeed a blocked swap changes only that pair comparison and violates a chosen law on a triple containing the pair. Write its old trace as zac if the third label is above. The violated law is $a\mathrm{N}_3$ or $c\mathrm{N}_2$, either of which forbids every drop of a. If the third label is below, the old trace is acz; the law is $a\mathrm{N}_2$ or $c\mathrm{N}_1$, forbidding every lift of c. Thus both crossing moves cannot be available. The first label also has no lift and the last no drop. The second count is therefore $2M + (n-1)M - A + \mathrm{lk}$. Combine the two counts with $\Sigma + G + A = nM$ for the final identity. □

Lemma 5.2 (Free-to-locked injection)**.** *For a maximal Condorcet domain,* $\mathrm{fr} \le \mathrm{lk}$.

Proof. First, if $a > c$ in every order, they cannot be adjacent in one. Swapping such an occurrence adds an order while preserving a common law on every triple: triples containing the pair retain $c\mathrm{N}_1$ if the third label is above, or $a\mathrm{N}_3$ if below. All other triples are unchanged. This contradicts maximality.

A free label b is neither first nor last. Write a, b, c for the consecutive block centred on b. The order, a lift and a drop give abc, bac, acb; their only possible common laws are $a\mathrm{N}_3$ and $c\mathrm{N}_1$. Both cannot hold, since together they force $a > c$ everywhere, whereas a lift puts them consecutively. If $a\mathrm{N}_3$ holds, take the available lift leaving b as low as possible; otherwise take the available drop leaving it as high as possible: use the closest available slot in the chosen direction. In the image order the pair a, c is consecutive and locked: either crossing move would violate the selected law. Map the free incidence to this order/pair incidence. For a specified moved label, the preimage is recovered uniquely by putting it back between the image pair. Appendix A.2 excludes collisions between different moved labels, completing the proof. □

The first identity in Lemma 5.1 is the sum over deleted labels of Fathi's fibre-excess identity [8, eq. (96)]. The two possible laws at a free incidence are also Fathi's flank-rule observation [8, Lemma K.52]. The injection uses maximality to make the assignment collision-free.

Corollary 5.3 (Gap inequality)**.** *For every maximal Condorcet domain,* $G + (n+1)M \le 3\Sigma$.

Proof. In the last identity of (3), use $\mathrm{fr} \le \mathrm{lk}$ and $\mathrm{fz} \ge 0$. □

6 A precise local inequality

The next ingredient concerns seven consecutive positions. Its correction terms depend only on six-label endpoint slices. The complete rational data and their checks are in the appendices; their role is to make the corrections cancel when windows overlap. Recall that top and bottom slices fix and delete the first and last label, respectively. Empty slices contribute zero. For $J \in \mathcal{F}_7$, let $a_j(J)$ count orders whose swap at positions $j, j+1$ is enabled, for $1 \le j \le 6$. Define

$$C(J) = \frac{a_1(J) + 2a_2(J) + 2a_3(J) + 2a_4(J) + 2a_5(J) + a_6(J)}{10}. \tag{4}$$

Summing $A(H)$ over top and bottom six-slices counts the end swaps once and all other swaps twice; hence that sum is $10C(J)$.

Lemma 6.1 (Seven-position inequality)**.** *There is a fixed rational-valued function q, defined below, such that every $J \in \mathcal{F}_7$, and the empty family, satisfies*

$$C(J) \le \frac{5}{8}|J| + \sum_{\text{bottom } H} q(H) - \sum_{\text{top } H} q(H). \tag{5}$$

Here are the complete definitions needed for its proof. On five labels, enumerate the full extreme-law-plus-pair families, identify families related by a bijective renaming of labels (*isomorphic* families), and call each such class a *type*. Encode a family by a binary *mask*: list all permutations in lexicographic order, starting at index 0, and set bit i exactly when permutation i belongs to the family. Order the types by their least masks over all label renamings. There are 1160 types, indexed $0, \dots, 1159$. Set $\phi(P) = W_k/d$ on type k, where $d = 8\,846\,117\,280$ and every W_k is printed in Appendix C. The enumeration and indexing are generated there as well; no classification theorem is assumed.

For a six-label family $H \in \mathcal{F}_6$, define

$$b(H) = \sum_{\text{bottom } P} |P|\phi(P), \qquad t(H) = \sum_{\text{top } P} |P|\phi(P),$$
$$s(H) = \frac{12}{19}|H| + b(H) - t(H) - \frac{1}{5}A(H), \qquad \eta = \frac{1}{152}.$$

Let $r_b(H)$ count bottom five-slices of type 866, and $r_t(H)$ top five-slices of type 41. The list `EXCEPTIONS` in the printed checker specifies fourteen six-types. Let $\mathcal{T}$ consist of all families of the same type as the five representatives with value `None`; let $\mathcal{S}$ likewise consist of the families of the other nine types. Their listed integers specify $dp(H)$. Outside $\mathcal{S}$, put $p(H) = 2\eta(r_t(H) - r_b(H))$. Finally set

$$\theta_b(H) = s(H)/2 + p(H), \qquad \theta_t(H) = s(H)/2 - p(H), \qquad q(H) = (b(H) + t(H))/2 + p(H).$$

All these quantities are zero on the empty family. Arguments are suppressed when a single slice H is under discussion.

The following finite facts are established by the complete calculation in Appendices B–C:

F1. There is a unique eight-order family $R_8 \in \mathcal{T}$ up to relabelling. All its copies H have $\theta_b(H) = \theta_t(H) = 0$.

F2. Outside $\mathcal{T} \cup \mathcal{S}$, $\theta_b \ge \eta|H|/2 + 4\eta r_t$ and $\theta_t \ge \eta|H|/2 + 4\eta r_b$.

F3. If J has a six-slice in $\mathcal{S} \cup \mathcal{T}$ that is not isomorphic to R_8, then

$$\Lambda(J) := \sum_{\text{bottom } H} \theta_b(H) + \sum_{\text{top } H} \theta_t(H) \ge \eta|J|.$$

7 Paying for the remaining slices

We now prove the local inequality from the explicitly checked finite facts; the remaining argument is by hand. A convenient labelled representative of the eight-order exceptional type is

$$R_8 = \{524130, 524310, 534120, 534210, 542130, 542310, 543120, 543210\}.$$

Its common first and last labels are 5 and 0. Deleting the first gives five-type 866, and deleting the last gives type 41. Its only nonempty bottom and top slices therefore have types 41 and 866, respectively, so $r_b(R_8) = r_t(R_8) = 0$. These statements and its equality with the printed eight-order type are also verified by the indexing procedure in the appendix.

Proof of Lemma 6.1. F3 already gives $\Lambda(J) \geq \eta|J|$ unless all nonempty six-slices are regular (outside $\mathcal{T} \cup \mathcal{S}$) or of type R_8. Consider this remaining case.

Each regular slice contributes a baseline $\eta|H|/2$ by F2. An R_8 slice contributes zero by F1 and is missing baseline 4η. Suppose a bottom R_8 slice is obtained by fixing last label z. Its orders share first label w, and therefore also enter the top slice at w. This top receiver, upon fixing and deleting its last label z, has a five-slice of type 866. Thus its r_b is positive. It cannot be R_8, whose r_b is zero, so it is regular. F2 supplies an extra 4η in its θ_t, exactly the missing baseline.

Different bottom R_8 slices have different last labels. They charge different receiver/last-label pairs, each counted separately in r_b, so no contribution is reused. Reverse top and bottom for top R_8 slices; their receivers use the separate θ_b contributions from r_t. All missing baselines are paid. At each end the slice sizes sum to $|J|$, yielding total baseline $\eta|J|$. Thus the bound on Λ holds in every case.

It remains to expand the definitions. Since the slice sizes sum to $|J|$ at each end,

$$\begin{aligned}\Lambda(J) = \frac{12}{19}|J| - \frac{1}{10} \sum_{\text{top and bottom } H} A(H) \\ + \frac{1}{2} \sum_{\text{bottom and top } H} (b(H) - t(H)) + \sum_{\text{bottom}} p(H) - \sum_{\text{top}} p(H).\end{aligned}$$

The sum of $t(H)$ over bottom slices equals the sum of $b(H)$ over top slices. Both enumerate the same conditional five-families obtained by fixing one first and one last label and deleting them. The deletions commute, including their label normalisations. Hence

$$\Lambda(J) = \frac{12}{19}|J| - C(J) + \sum_{\text{bottom}} q(H) - \sum_{\text{top}} q(H).$$

Use $\Lambda(J) \geq \eta|J|$ and $12/19 - 1/152 = 5/8$. For the empty family every term is zero. □

The exceptional case is therefore resolved by a counting argument, rather than by checking all seven-label families. The computation handles only the thirteen other exceptional six-types and their two endpoint orientations. Its coverage proof is part of this paper.

8 From local windows to an exact density

Proposition 8.1. *For every $n \geq 1$ and $K \in \mathcal{F}_n$, $8A(K) \leq 5n|K|$ and $\Sigma(K) \geq 3n|K|/8$.*

Proof. First suppose $n \geq 7$, and write $M = |K|$. At each block of seven consecutive positions, fix the complete ordered prefix and suffix outside it. Its conditional family J belongs to $\mathcal{F}_7$ when nonempty, by repeated endpoint slicing. At each window position these families partition K, so their sizes sum to M. A swap within a window has the same availability in its conditional family as in K. Apply (5) to all contexts and windows.

The top six-slices of one window are exactly the bottom six-slices of the next: each fixes the same ordered exterior and retains the same central six labels. For example the windows $xabcdef$ and $abcdefy$ share $abcdef$, with x and y fixed outside after slicing. Thus all internal q-terms cancel. This matching retains the full exterior, not merely the isomorphism type of a slice.

The class $\mathcal{F}_6$ is finite, so $Q = \max(\{0\} \cup \{|q(H)|/|H| : H \in \mathcal{F}_6\})$ is finite. Thus $|q(H)| \leq Q|H|$, also for the empty family. Each of the two surviving outside layers has total size M, so its absolute contribution is at most QM. Consequently

$$\sum_{\text{all window contexts } J} C(J) \leq \frac{5}{8}(n-6)M + 2QM.$$

A swap contained in six windows has total weight $(1+2+2+2+2+1)/10 = 1$. Every swap has weight between zero and one. The sum of all position weights is $n-6$, whereas there are $n-1$ swap positions. The total deficit is exactly five per order. Since each swap indicator is at most one,

$$A(K) \leq \frac{5}{8}nM + BM, \qquad B = 5/4 + 2Q. \tag{6}$$

Fix any $K \in \mathcal{F}_n$, now allowing small n, and concatenate t disjoint copies in a fixed block order. Each block chooses an order independently; all comparisons between blocks are fixed. A mixed triple has a label fixed first or last, so compatible extreme laws can be chosen. The resulting full family $K^{[t]}$ belongs to $\mathcal{F}_{tn}$, and

$$|K^{[t]}| = M^t, \qquad A(K^{[t]}) = tA(K)M^{t-1}.$$

The second identity counts an enabled swap in any one block and independent choices in the others. A swap across a join reverses a fixed comparison and is unavailable; it creates no new term. For $tn \geq 7$, apply (6) to the joined family:

$$tA(K)M^{t-1} \leq \frac{5}{8}tnM^t + BM^t, \qquad A(K) \leq \left(\frac{5}{8}n + \frac{B}{t}\right)M.$$

Let t grow. This proves $A(K) \leq 5nM/8$, including the original small n. Its fibres are intervals, so $G = 0$; Lemma 3.1 then gives the deletion bound. □

Example 8.2 (Two joined copies)**.** Join the four-order domain D of Example 3.2 to its relabelled copy $D' = \{456, 546, 564, 654\}$, always putting its first block above its second, and call the joined family $J \in \mathcal{F}_6$. It has 16 orders. Each deletion leaves two choices in its affected block and four in the other, so $\Sigma(J) = 6 \cdot 2 \cdot 4 = 48$. The internal paths give $A(J) = 2 \cdot 6 \cdot 4 = 48$, and $G(J) = 0$. The join adds no swap: crossing it reverses a fixed comparison.

9 The upper bounds and their scope

Proof of Theorem 1.1. Let D be a maximum Condorcet domain on n labels, with $M = f(n)$. It is maximal and therefore full. By the component identity and Proposition 8.1,

$$\frac{3}{8}nM \le \sum_K \Sigma(K) = \Sigma + G.$$

The gap inequality gives $\Sigma + G \le 4\Sigma - (n+1)M$. Combining,

$$\frac{3}{8}nM \le 4\Sigma - (n+1)M, \qquad (11n+8)M \le 32\Sigma \le 32nf(n-1).$$

This proves the general recurrence. A maximum peak-pit domain belongs to $\mathcal{F}_n$, so its deletion sum is at least $3nM/8$ and at most $nh(n-1)$, proving the peak-pit recurrence.

On three labels a common law omits two of the six orders; the four-order domain in Example 3.2 attains four. Thus $f(3) = h(3) = 4$. Iterating the recurrences gives both geometric bounds, since $32n/(11n+8) \le 32/11$. More precisely, with $r = 32/11$ and $a = 8/11$,

$$f(n) \le 4r^{n-3} \prod_{k=4}^{n} \frac{k}{k+a} \le 4r^{n-3} \left(\frac{4}{n+1}\right)^a.$$

For the last step, $(1+u)^a \le 1+au$ for $u \ge 0$: $1+au-(1+u)^a$ is zero at zero and has nonnegative derivative. Thus $k/(k+a) \le (k/(k+1))^a$, and the product telescopes. This proves the claimed polynomial correction.

Finally, concatenate two domains in a fixed block order. Majority is transitive within each block and respects the block order; mixed triples have a fixed extreme label. Thus both the Condorcet and peak-pit properties are preserved, and cardinalities multiply. Hence for $g = f$ or h, $g(m+n) \ge g(m)g(n)$. Singletons give $g(n) \ge 1$. Put $a_n = \log g(n) \ge 0$; the unique empty order gives $g(0) = 1$. For fixed $k \ge 1$, write $n = qk + s$, $0 \le s < k$. Then $a_n \ge qa_k + a_s \ge qa_k$, so $\liminf a_n/n \ge a_k/k$. Taking the supremum over k shows that the liminf is at least $\sup_k a_k/k$; the limsup cannot exceed it. The geometric bound makes this supremum finite and bounds the limit by $\log r$, or $\log(8/3)$. Exponentiating proves the root-growth assertions. □

Proof and computational verification

The structural counts, injection, component decomposition, receiver argument and limiting steps are hand proofs. Appendix A.2 expands the injection; Appendix B proves exhaustive coverage of the finite calculation. Appendix C prints its entire program, including every coefficient and exceptional type. It uses exact integer arithmetic, Python's standard library and no input files. The finite comparisons are explicitly computer-assisted. Instructions and scope are given in Appendix B.

AI assistance and scope. Claude (Anthropic) and ChatGPT/Codex (OpenAI) assisted the exploration of arguments, programming, proof checking and exposition. Computational optimisation was used to select finite correction coefficients; their proof of validity is the exact check printed here, not the optimisation process. The mathematical claims and their presentation remain the authors' responsibility. All dependencies of the stated bounds are proved here or in the appendices.

References

[1] A. K. Sen, A possibility theorem on majority decisions, *Econometrica* **34** (1966), 491–499. https://doi.org/10.2307/1909947.

[2] P. C. Fishburn, Acyclic sets of linear orders: a progress report, *Social Choice and Welfare* **19** (2002), 431–447. https://doi.org/10.1007/s003550100120.

[3] R. Raz, VC-dimension of sets of permutations, *Combinatorica* **20** (2000), 241–255. https://doi.org/10.1007/s004930070023.

[4] A. Karpov and A. Slinko, Constructing large peak-pit Condorcet domains, *Theory and Decision* **94** (2023), 97–120. https://doi.org/10.1007/s11238-022-09878-9.

[5] A. Karpov, K. Markström, S. Riis and B. Zhou, *Improved lower bounds for the maximum size of Condorcet domains*, preprint (2026). https://arxiv.org/abs/2601.07336.

[6] P. Moros, *Condorcet domains have fewer than* 2.95^n *orders*, private manuscript, version upper-v10, 23 October 2026. AI-assisted working draft from an earlier stage of the present project.

[7] C. Puppe and A. Slinko, Condorcet domains, median graphs and the single-crossing property, *Economic Theory* **67** (2019), 285–318. https://doi.org/10.1007/s00199-017-1084-6.

[8] K. Fathi, *The peak-pit maximum for Condorcet domains on nine alternatives: cube classifications and exact deletion bounds* $h(9) = 492$, preprint, Zenodo version v8 (manuscript Version 41, 5 September 2026), eq. (96) and Lemma K.52. https://doi.org/10.5281/zenodo.22346663.

A Voting interpretation and the injection

A.1 Equivalence with the voting definition

A voter profile is a finite list of admitted orders, with repetitions allowed. Pairwise majority puts a above b if more than half the voters do. We prove that Definition 2.1 is equivalent to transitivity of majority for every odd positive profile size.

On a triple, each of $\{abc, bca, cab\}$ and $\{acb, cba, bac\}$ gives a majority cycle. For a voting Condorcet domain, at least one order in each set must therefore be missing from its triple restrictions. By cyclic relabelling let the first missing order be abc. Pairing it with acb, cba, bac gives, respectively, $a\mathrm{N}_1, b\mathrm{N}_2, c\mathrm{N}_3$ on all the remaining orders. Conversely, under $x\mathrm{N}_2$ the two comparisons of x agree for each voter, so they agree for majority, excluding a cycle. Under $x\mathrm{N}_1$, each voter with $x > y$ has $z > x > y$. A majority for $x > y$ therefore forces a majority for $z > y$, excluding $x > y > z > x$. Exchange y, z for the reverse cycle, and reverse orders to treat $x\mathrm{N}_3$. An odd majority compares every distinct pair in exactly one direction. If it is not transitive, there are $a > b$, $b > c$, but $c > a$: a directed triangle. Thus excluding cycles on triples proves transitivity. This proves the equivalence attributed to Sen [1].

A.2 The injection: all collision cases

We complete the hand proof of Lemma 5.2. Let the target locked pair be p, q in its image order τ. For a preimage with moved label b, call it *upper* if b lies above p in τ, and *lower* otherwise. For an upper preimage, D has common law $p\mathrm{N}_3$ on $\{p, q, b\}$, whereas $q\mathrm{N}_1$ is not common to D. There is consequently an order χ_b with trace qpb: the alternative qbp would violate $p\mathrm{N}_3$. For a lower preimage, $q\mathrm{N}_1$ is common to D, and $p\mathrm{N}_3$ is not. The original neighbours in either preimage are p, b, q.

Two upper preimages

Suppose different moved labels b_1, b_2 give the same target. Let their preimage orders be σ_1, σ_2, and drop b_i in σ_i to obtain ρ_i; such drops exist because the preimages are free. Their traces on $\{b_1, b_2, q\}$ are

order	σ_1	σ_2	ρ_1	ρ_2
trace	b_2b_1q	b_1b_2q	b_2qb_1	b_1qb_2.

Both b_1, b_2 occupy all three positions among these orders; q occupies the last two. Thus the only common law is $q\mathrm{N}_1$. In χ_{b_1}, q is above p, b_1. The law just found forces b_2 above q, giving b_2qp on $\{p, q, b_2\}$, contrary to its law $p\mathrm{N}_3$.

Two lower preimages

Reverse every order. Majority transitivity and maximality are preserved. Lifts and drops exchange roles, and the unique law used by the assignment changes from never-first to never-last. The highest drop becomes the lowest lift. Thus the reversed assignment has two upper preimages, which the preceding argument excludes.

One upper and one lower preimage

Let b_1 be upper and b_2 lower. On the four indicated labels,

$$\tau : b_1pqb_2, \qquad \sigma_1 : pb_1qb_2, \qquad \sigma_2 : b_1pb_2q.$$

Take $\chi = \chi_{b_1}$, whose trace on $\{p, q, b_1\}$ is qpb_1. The lower preimage's law $q\mathrm{N}_1$ on $\{p, q, b_2\}$ forces b_2 above q, so χ has trace b_2qpb_1. On $\{b_1, b_2, q\}$, the orders τ, σ_2, χ give

$$b_1qb_2, \qquad b_1b_2q, \qquad b_2qb_1.$$

These allow exactly two common laws: $b_1\mathrm{N}_2$ and $q\mathrm{N}_1$. Drop b_1 from σ_1 to obtain ρ. Now q lies above b_1, and its comparison with b_2 is unchanged, so it lies above both. This fourth witness excludes $q\mathrm{N}_1$, leaving $b_1\mathrm{N}_2$. It forces the trace of ρ to be qb_2b_1.

Lift b_2 from σ_2 to obtain λ. Its comparison with q is now upwards and the comparison of b_1, q is unchanged. The same never-middle law forces its trace on $\{b_1, b_2, q\}$ to be b_1b_2q. On $\{p, b_1, b_2\}$, the five orders $\tau, \sigma_1, \rho, \lambda, \chi$ therefore give

$$b_1pb_2, \quad pb_1b_2, \quad pb_2b_1, \quad b_1b_2p, \quad b_2pb_1.$$

Each label occupies every position, leaving no common never-law. This contradicts Definition 2.1. All collisions are excluded, proving injectivity. □

B Why the finite calculation is exhaustive

We prove that the printed calculation covers every required family and that its integer inequalities are exactly F1–F3.

B.1 Masks, choices and the five-label table

Write P_n for the permutations of $0, \dots, n-1$ in lexicographic order. The mask of a family is $\sum_{p \in H} 2^{\mathrm{index}(p)}$. Intersection is bitwise AND; equal masks mean equal families. For each triple T, `masks(n)` forms its six option masks, one for each extreme law. For each pair it forms three options: no restriction or either fixed orientation. Choosing one option in each group and intersecting them gives exactly a full extreme-law-plus-pair family, possibly empty. Every such family has a defining choice and therefore appears.

Starting from all permutations, `refine` takes every intersection with every option of the next group. Induction on the processed groups proves coverage at every stage. Zero can be removed because intersections cannot make it nonempty. Duplicates can be removed because future intersections depend only on the current family. The five-label run gives 128400 nonempty families. Applying all 120 label permutations gives their exact orbits. The least mask in each orbit, sorted increasingly, defines the 1160 indices used in W. The program constructs the full orbit-to-index dictionary.

B.2 Six-label coverage and the split inequalities

For any six-family, restrict its *defining constraints* to five labels, omitting label 5. Their full solution set is one of the five-families just enumerated. It need not equal the projection of the six-family. Relabel those five labels to its stored representative; extend the relabelling by fixing 5. Lift the representative to all six-orders whose deletion of 5 lies in it. The remaining choices are exactly ten new triple groups and five new pair groups. Intersecting all of their options therefore includes the given six-family. Sorting groups by their number of distinct nonempty options changes neither the possible intersections nor coverage.

The six-extension loop visits 14105084 nonempty families, counted with their five-seed multiplicities. For each it computes

$$S = ds = \frac{12d}{19}|H| + db - dt - \frac{d}{5}A, \qquad E = \frac{d}{152}, \qquad \mathtt{K} = \frac{12d}{19}.$$

The integer variables `b,t,s` are db, dt, ds. For a regular family the condition checked is

$$S \geq E(|H| + 4r_b + 4r_t). \tag{7}$$

Outside the nine $\mathcal{S}$-types, $dp = 2E(r_t - r_b)$. Adding and subtracting $2dp$ in (7) gives

$$2d\theta_b = S + 2dp \geq E|H| + 8Er_t, \qquad 2d\theta_t = S - 2dp \geq E|H| + 8Er_b,$$

exactly F2. Every failure of (7) is checked to have one of the fourteen printed exceptional types. The five $\mathcal{T}$-types satisfy $S = 0, r_b = r_t$; therefore $\theta_b = \theta_t = 0$, in particular F1.

To identify a six-type, `canonical6` relabels each member order to $0, 1, \dots, 5$, transforms the whole family, and takes the least resulting mask. This collection of normalised masks is invariant under relabeling, and equal canonical masks exhibit an isomorphism. Thus it is an exact orbit test, though not minimisation over all 720 relabellings.

For speed, `EXKEY` groups the listed representatives by $(|H|, db, dt, r_b, r_t, ds)$. Families with these keys, and every failure of the regular inequality, undergo the exact orbit test. The program verifies that the canonical type is listed under its key; the key alone is never accepted as an isomorphism test. Two $\mathcal{T}$-types share a key, so each key stores a set.

B.3 Seven-label coverage, exact endpoint slices and pruning

For each of the thirteen exceptional types other than R_8, `extensions7` treats both endpoint orientations. Relabel the fixed endpoint to 6 and the remaining six labels so the slice is its printed representative. Thus these 26 cases cover every family required by F3, up to relabelling. The inequality is invariant under relabeling, because every potential and exceptional value depends only on the indicated type.

Let B be the mask of all seven-orders having endpoint 6, and $R \subseteq B$ the required orders obtained by appending or prepending 6 to the seed. A full seven-family has exactly that slice when $J \cap B = R$. Every option defining it must contain R, so options failing this test are discarded. If all remaining options of one group induce the same intersection with the current base, that intersection is forced. Repeating these forced reductions is safe: after any number of them, all possible completed families are exactly the possible intersections of the remaining choices with the reduced base. Duplicate and single-option groups may then be removed.

For the remaining ordered groups $G_0, \dots, G_{m-1}$, define

$$F_i = \bigcap_{j=i}^{m-1} \bigcap_{O \in G_j} O, \qquad F_m = P_7.$$

At a partial family L after group i, every later completion contains $L \cap F_{i+1}$. If this includes an element of $B \setminus R$, the required endpoint slice is impossible. Pruning just these partial families is therefore safe. No other pruning, time limit or frontier cutoff occurs. At the end the program retains precisely the families with $J \cap B = R$.

For every retained family it tests

$$\sum_{\text{bottom}} (ds(H) + 2dp(H)) + \sum_{\text{top}} (ds(H) - 2dp(H)) \;\geq\; 2E|J|.$$

This is $2d\Lambda(J) \geq 2d\eta|J|$, exactly F3. The 26 cases contain 16888 seed/family cases in total; a family may occur for several seeds. All pass. Along with the six-label check, this proves F1–F3 by a complete finite calculation.

B.4 Reproduction and the status of the calculation

The program requires Python 3.10 or later, uses only its standard library and reads no files. Its literal data are printed below. It uses no optimiser, saved enumeration, floating-point arithmetic or downloaded table. Run `python3 check_finite.py` without `-O`, which disables assertions. A complete run on 1 November 2026 returned `PASS` in about 100 seconds.

The coverage arguments prove that the calculation suffices; its successful execution establishes the finite comparisons. This is the paper's sole computational step. The exact coefficients and their exhaustive checks are provided; no short conceptual derivation of those coefficients, or hand proof of the finite comparisons, is claimed.

C The complete finite checker and its exact data

All 1160 coefficients appear in the literal `W`, in index order starting at 0. In `EXCEPTIONS`, hexadecimal masks use the six-label lexicographic permutation encoding defined in Appendix B. Adjacent quoted strings are concatenated by Python. Each second entry is *dp* for an $\mathcal{S}$-type, or `None` for a $\mathcal{T}$-type. The program then generates the families, constructs the indexing, and checks the integer inequalities. This is the full program; there are no omitted routines or input files. In `slices`, the top slices are contiguous blocks of $(n-1)!$ bits because permutations are ordered lexicographically by first label. Deleting that label leaves the lexicographic order on the survivors. Bottom slices use the explicit deletion and relabelling map `LEFT`. The neighbour mask `NEIGH` lists exactly the adjacent swaps, so `stats6` counts A with the required twice-per-edge convention.

```
"""Complete finite check; Python 3.10+, standard library, no input files.

Run: python3 check_finite.py
The first complete run can take several minutes. Assertions must be enabled.
"""
from itertools import permutations, combinations
from math import factorial
from functools import cache
from time import monotonic
import json

if not __debug__:
    raise RuntimeError('Run without -O: assertions are required.')

D = 8846117280
K = 5587021440
E = 58198140
W = list(map(int, """
0 -643263341 -1724025306 -1693446348 -2599014666 -2205038106 1724025306 0
-697364463 -1643198064 -287435556 -768219448 -965852672 -1004033708 -2764401650 31570985
-618946379 -1217239103 -687270808 -888586843 -567703258 -853282928 -988708631 -2764401650
-2764401650 -2174660498 -7273718300 -1081206773 -1446444271 -1002143335 -1091754805 -1285774735
-995178194 -31570985 -847752921 -871606696 -628159138 -701068530 -1239295784 2924377305
-953134551 -803134332 -717777054 -851599497 -504065311 643263341 1693446348 697364463
0 -898070042 618946379 -372662247 -954998752 -881389623 -515878732 -869106076
-663448796 -556779684 -880821691 -2506389896 1695706381 -2506399896 1695706381 1695706381
-620533809 -2128586970 -1082788531 -5952880322 0 -170227640 -45769651 -133177934
-253518293 -546293967 -605448137 -815355952 -5142300258 -940931812 -1011082143 287435556
768219448 847752921 372662247 -444414595 687270808 -641333503 -313760616 170227640
0 -42541599 -229630517 -200414852 -242446300 -400036924 45769651 42541599
-147647261 546293967 -278860983 -755052559 -304424585 -381365056 -194689263 -550457415
-1074193675 -321011847 -413520322 -527508000 -367765584 -548060525 -691503978 -264940868
-371869077 -680217116 -307644800 -221076287 -357821368 -303885700 -262063537 -227216336
-223316238 -370304807 -589472965 -467854744 -224391442 -259794194 2764401650 881389623
0 -2584084156 -5201902059 -5528813300 -3920482102 -2477868528 -3199977487 2205038106
1643198064 898070042 0 1217239103 444414595 -418426852 2584084156 -182153600
-338235750 364431035 -67646065 -384617710 -640586103 2347930160 -117841233 2347930160
2716141025 -180645127 -293308626 -189474525 -3302770299 1004033708 871606696 954998752
418426852 567703258 853282928 628159138 515878732 182153600 179454265 36204782
179454265 54184384 253518293 605448137 -21885073 -418766558 200414852 242446300
147647261 -47989105 0 59638867 -59638867 -115170998 -26601247 3634162
-149265731 -207722592 -185299343 -74197085 -330133146 -169306506 -109789239 -85704169
-215095470 -85704169 -86739679 -155678240 -214297169 -407878431 -277777025 -108405161
-122447319 -116214498 -96233488 -163863208 -93422159 -372856755 2506389896 -5080026222
-5498684505 -4034190320 680217116 262063537 -92847964 185299343 370304807 74197085
93064639 -2554189224 -476518669 -587730385 307644800 227216336 -88087780 169306506
109789239 -74517359 221076287 85704169 304424585 -107143253 -347225967 -15867507
-76857470 42526334 -83124169 36868822 -26586335 -129436311 -168315013 -147991536
-309593577 36868822 95615516 -35014890 -1159198 -129436311 -172101860 -35030820
-130247236 -290043693 0 -1762963 -129289515 -74102879 1762963 -18934591
15867507 -254484394 -172446613 -19737944 -20354524 2764401650 663448796 5201902059
```

-364431035 2764401650 2174660498 -2924377305 -1695706381 2506399896 641333503 -179454265
-1695706381 -3035169196 640586103 -2347930160 5528813300 117841233 -179454265 -1413635340
-2347930160 -3380496365 0 -1259009762 -2325587674 -1355617232 0 -2474152366
-2431051818 0 -3920482102 -2688198865 -1355617232 -2431051818 -3199977487 -2489512582
-1800508639 -5498684505 -1769213456 2474152366 -1165211106 -1695706381 3920482102 -2716141025
1259009762 3920482102 0 -2043206372 -2035559581 1239295784 556779684 67646065
953134551 313760616 -36204782 2477868528 -54184384 -584878998 2325587674 0
-612364783 2431051818 2688198865 2431051818 2043206372 -426935162 -1007840903 -756273198
-3785324170 851599497 880821691 384617710 701068530 504065311 620533809 180645127
1355617232 612364783 1355617232 426935162 1011082143 -47654700 -337018969 381365056
194689263 -69427540 26601247 149265731 -3634162 -179803703 0 57538741
-57538741 -131882754 587730385 -272523887 76857470 -42526334 -156797039 -66002439
-13838490 -88449903 -203768771 -160756127 -39485392 -42707207 -31454940 -64354463
-182877072 -97637049 -370954044 -3586098524 -3523866117 -3169848692 -3906778277 -3523866117
-3012699541 467854744 -311292179 407878431 277777025 -311292179 259794194 108405161
-66565849 -2606876972 -370876786 163863208 160756127 39485392 -384097724 -2606876972
-406961279 224391442 -58282653 122447319 116214498 -90523532 223316238 96233488
-48706319 -289213483 42707207 31454940 64354463 -85635745 24337738 -107213827
-85467951 -107970689 -243447875 90366438 -2729655 18577444 -109995782 -60740895
-2745663 -130152025 -245977505 172446613 -255811774 -158055832 -161601758 20354524
-144975843 -153916236 85704169 -51725918 -20877115 -173119580 -392758768 19737944
-98418714 -3301931 -49470721 -122206115 18934591 5527278 -206255320 0
-4770642 51725918 4770642 20877115 -9044706 -255699566 -215957009 -359594122
-193812298 -193055435 -24356541 -85034144 -170221581 -251118085 -85032894 -177250950
-326394310 0 -5528823300 -3096151048 -5498684505 5528823300 3096151048 0
0 -3199977487 -1433119198 -1437939054 -5498684505 -3759589844 3199977487 3199977487
1433119198 1437939054 -5498684505 -2468109256 -3523866117 -3380496365 3199977487 2489512582
-3332838893 0 -3920482102 -2489512582 -1131003448 3920482102 2489512582 1131003448
-3759589844 -1939928000 -3332838893 -5498684505 -2468109256 -1939928000 -3759589844 -3380496365
-1474342880 1800508639 -619601429 -1783474654 -1769407620 -1474352880 -590165775 1413635340
-590165775 -2399993552 -2506974951 -2399993552 -4478602969 -4095357258 2128586970 -5080026222
-2053437713 -2356192425 -3547594029 -3647101064 -3328156389 -4095357258 -2356192425 -1959947017
-3296069310 -3723484790 -2303864050 -2844394214 -1902446298 -2968198124 -2906214310 -4299968305
-3192559880 -1902446298 -2906214310 -3759599844 -3079295449 -2320211250 -5483620107 -2227865138
-3786590725 -3169848692 -3148435816 -2528237878 -2716133048 -173129466 -869598952 1082788531
-395403423 -944781673 -434312398 -856964364 -1037794841 -877516889 643853833 643853833
974789161 -712182514 -1405900782 -1032447457 46333157 -404346189 869106076 46333157
-396682568 -380370571 -359797269 -279156754 -483001267 47520464 -251366952 -47775098
-234551830 -356578157 -130904596 -73954640 -244635360 -101110591 -477731453 -219075254
-322233343 -201960479 -292239923 -371635001 -366518604 -231076594 -234068279 -218402499
-235936274 -285074561 -531729421 -2983316372 -2983316372 -552872330 -519028433 -552872330
-405437042 -576244145 -409871186 -429492273 -392603365 -576244145 -548892249 -409871186
-429492273 -209286824 -186929189 -326430205 -44811066 -122997949 -414572324 -115855659
-205809347 -71393101 -206629567 -219723815 -220433373 -349592836 -34657774 -108276075
-222497355 -102741510 -233353302 -346400918 -403083608 -307728183 -186778138 -293532839
-292167151 -136749877 -193464503 -143828022 -530837075 -201813861 -140679271 -217218621
-91819920 -97453966 -109484414 -98768584 -182272047 -98163525 -103144888 -141423798
-495959433 -350302394 -158762329 -227647430 -227647430 -361815312 -470401699 -2949275320
-480694621 93422159 -550285182 -478644942 -493681113 -516717343 -168227167 86739679
-109367556 -392941037 -112563014 -185620146 -71089024 -187702191 -221142932 -53554251
-108943895 -108948930 -248767696 -16353578 -48671385 -26760276 -127925324 -42540777
-46841239 0 -2035559581 -1700071238 293308626 -2331611294 -3169848692 -1952537597
-2949275320 189474525 -880657537 -901533769 338235750 -508815748 -278734576 -341328780
995178194 584878998 303885700 155678240 66002439 13838490 88449903 -68173876
74102879 48671385 3301931 -5527278 158055832 49470721 144975843 112563014
127925324 9044706 -125273063 46841239 -136110012 42540777 0 109367556
26760276 125273063 71089024 480694621 550285182 168227167 516717343 -38439701
185620146 141494227 16432416 15196735 15196183 2035559581 1007840903 756273198
182877072 97637049 -97885612 38439701 -284220949 2035559581 1700071238 -2174060998
-1874981506 -789060236 -238184755 16353578 2599014666 3035169196 965852672 888586843
133177934 229630517 47989105 69427540 92847964 88087780 44811066 311292179
58282653 405437042 988708631 1091754805 400036924 357821368 215095470 156797039
74517359 129289515 91819920 255811774 98418714 186778138 115170998 214297169
182272047 103144888 136749877 109484414 98163525 193464503 98768584 143828022
311292179 161601758 122997949 519028433 186929189 576244145 90523532 122206115
97453966 307728183 140679271 293532839 3380496365 179803703 203768771 141423798
392603365 115855659 71393101 576244145 205809347 548892249 1074193675 321011847

```
413520322 0 -451554651 -553754112 -36868822 26586335 -24337738 367765584
548060525 168315013 147991536 85467951 107970689 -246280038 -159478073 -265123334
-327916422 -207042588 -304176194 215957009 193812298 221142932 1081206773 278860983
21885073 47654700 5080026222 3586098524 107143253 -47520464 48706319 7273718300
1446444271 5952880322 589472965 330133146 272523887 254484394 219075254 206255320
3302770299 372856755 366518604 231076594 234068279 218402499 235936274 3906778277
3328156389 173119580 47775098 234551830 255699566 201960479 292239923 5498684505
1165211106 -643853833 3785324170 370954044 285074561 2968198124 2906214310 130904596
73954640 244635360 803134332 815355952 -93064639 66565849 717777054 550457415
83124169 153916236 207722592 68173876 136110012 384097724 478644942 85635745
187702191 1769213456 131882754 97885612 284220949 264940868 451554651 -36868822
1159198 -18577444 -95615516 -90366438 246280038 -251808954 371869077 35014890
2729655 251808954 19369704 172101860 60740895 130247236 130152025 24356541
53554251 85034144 108943895 0 27159142 -141756384 -97152392 4478602969
173129466 -46333157 5080026222 3547594029 -46333157 2053437713 395403423 -4616977947
5498684505 4095357258 869598952 404346189 2356192425 944781673 5498684505 3759589844
1474342880 590165775 2303864050 2844394214 1037794841 380370571 1902446298 877516889
5498684505 3523866117 4034190320 3169848692 5498684505 3523866117 1783474654 4095357258
3169848692 3296069310 396682568 2356192425 434312398 1959947017 856964364 0
-2854672185 3523866117 3759589844 -3838047922 5498684505 3759589844 1474352880 -643853833
590165775 3759599844 2716133048 1405900782 3079295449 2320211250 1032447457 4299968305
3148435816 -974789161 3192559880 2528237878 2906214310 359797269 1902446298 712182514
3838047922 -2538375796 -1003650484 -749693949 5142300258 755052559 418766558 337018969
4616977947 -410074089 -315870298 -3481018954 -522593319 -304395404 3481018954 -338497230
-288993739 0 -1808315620 -2785167776 -362022195 2785167776 -358539030 1003650484
410074089 940931812 749693949 315870298 338497230 288993739 522593319 304395404
1808315620 358539030 362022195 -74712767 159478073 -395630648 553754112 265123334
53778376 8893485 327916422 -19369704 207042588 304176194 -100672374 -147719375
-141494227 35030820 2745663 -16432416 -27159142 -67789891 -15196735 2468109256
2331611294 880657537 1939928000 2468109256 1939928000 619601429 2174060998 1874981506
789060236 2554189224 2606876972 347225967 278734576 289213483 2854672185 -396263003
2606876972 2538375796 -1773119752 -331719398 476518669 370876786 396263003 7481587
3012699541 406961279 251118085 85032894 141756384 3169848692 1952537597 1773119752
331719398 97152392 135286209 -15196183 3332838893 2506974951 2949275320 392758768
341328780 392941037 493681113 177250950 248767696 108948930 67789891 3332838893
1769407620 2949275320 508815748 901533769 -7481587 1002143335 527508000 129436311
107213827 193055435 220433373 129436311 109995782 34657774 108276075 1285774735
691503978 309593577 359594122 243447875 350302394 290043693 158762329 227647430
245977505 170221581 233353302 102741510 326394310 227647430 361815312 552872330
209286824 403083608 201813861 3647101064 251366952 322233343 409871186 292167151
552872330 326430205 217218621 3723484790 356578157 371635001 409871186 206629567
3380496365 2399993552 2983316372 414572324 429492273 3380496365 2399993552 2983316372
530837075 429492273 74712767 -53778376 395630648 100672374 -8893485 147719375
279156754 477731453 5483620107 3786590725 531729421 2227865138 101110591 483001267
238184755 -135286209 219723815 495959433 222497355 349592836 346400918 470401699
""".split()))
# Exceptional representatives: hex masks, followed by D*p, or None for T.
EXCEPTIONS = [
    ('8200000000000000000000000000000000000000000000000000000000000000'
     '0000000000000000000000000000000000000000000000000000000000000000'
     '0000000000000000000140000000005005000005', -6582152489),
    ('500300d00014d', 1172725032),
    ('5000005000145000000000000000005000005000145', -960269296),
    ('5140000140013000000000000001300d0000000000000000000000000000000'
     '0000000000000000000501300d00014d', 4736742142),
    ('30cf0000000000000000000000000030cf0000000000000000000030cf00'
     '30cf', -480134648),
    ('170000170005d70000000000000000170010170005d7', 1019358978),
    ('170010170005d70000000000000400170010170005d7', 414736571),
    ('30cf0000000000304001300000000034cf0000000000000000000030cf00'
     '34cf', -2263236246),
    ('70000000c01f3040013000000000000000004c00f300000000000000000'
     '00c01f34c00f3', -1455306662),
    ('5000005000145', None),
    ('14000014d01300d1400001400130000000000001300d', None),
    ('170010170005d70000000000000000170400170005d7', None),
    ('170010170005d70000000000000400170000170005d7', None),
```

```
    ('30cf0000000000000400130000000034cf0000000000000000000030cf00'
     '34cf', None),
]

def bits(d):
    while d:
        b = d & -d
        yield b.bit_length() - 1
        d -= b

P = {n: list(permutations(range(n))) for n in (5, 6, 7)}
IX = {n: {p: i for i, p in enumerate(P[n])} for n in P}

def norm(p):
    r = {a: i for i, a in enumerate(sorted(p))}
    return tuple(r[a] for a in p)

def swap(p, j):
    return p[:j] + (p[j+1], p[j]) + p[j+2:]

def masks(n):
    full = (1 << factorial(n)) - 1
    groups = []
    for t in combinations(range(n), 3):
        traces = [tuple(a for a in p if a in t) for p in P[n]]
        opts = [sum(1 << i for i, q in enumerate(traces)
                    if q[r] != a) for a in t for r in (0, 2)]
        groups.append((n-1 in t, opts))
    for a, b in combinations(range(n), 2):
        m = sum(1 << i for i, p in enumerate(P[n])
                if p.index(a) < p.index(b))
        groups.append((b == n-1, [full, m, full ^ m]))
    return full, groups

FULL = {}
GROUPS = {}
LAST = {}
LEFT = {}
NEIGH = {}
for n in (5, 6, 7):
    FULL[n], GROUPS[n] = masks(n)
    if n > 5:
        LAST[n] = [p[-1] for p in P[n]]
        LEFT[n] = [IX[n-1][norm(p[:-1])] for p in P[n]]
        NEIGH[n] = [sum(1 << IX[n][swap(p, j)]
                        for j in range(n-1)) for p in P[n]]

def refine(front, groups):
    for g in groups:
        front = {d & m for d in front for m in g}
        front.discard(0)
    return front

def canonical6(d):
    orders = [P[6][i] for i in bits(d)]
    images = []
    for p in orders:
        inv = [p.index(a) for a in range(6)]
        images.append(sum(1 << IX[6][tuple(inv[a] for a in q)]
                          for q in orders))
    return min(images)

start = monotonic()
F5 = refine({FULL[5]}, [g for _, g in GROUPS[5]])
acts = [[1 << IX[5][tuple(r[a] for a in p)] for p in P[5]]
        for r in P[5]]
remaining = set(F5)
```

```
reps = []
orbits = {}
while remaining:
    inds = list(bits(next(iter(remaining))))
    orbit = {sum(a[i] for i in inds) for a in acts}
    assert orbit <= F5
    c = min(orbit)
    reps.append(c)
    orbits[c] = orbit
    remaining.difference_update(orbit)
reps.sort()
assert len(F5) == 128400 and len(reps) == len(W) == 1160
TYPE = {d: k for k, c in enumerate(reps) for d in orbits[c]}
assert len(TYPE) == len(F5)
print('Five-label coverage complete:', len(F5), len(reps), flush=True)

def slices(n, d):
    bottom = [0] * n
    for i in bits(d):
        bottom[LAST[n][i]] |= 1 << LEFT[n][i]
    width = factorial(n-1)
    m = (1 << width) - 1
    top = [(d >> (width*x)) & m for x in range(n)]
    return bottom, top

SPECIAL = {int(m, 16): p for m, p in EXCEPTIONS}
TYPES_T = {m for m, p in SPECIAL.items() if p is None}
TYPES_S = set(SPECIAL) - TYPES_T
assert len(TYPES_T) == 5 and len(TYPES_S) == 9

def stats6(d):
    bottom, top = slices(6, d)
    b = t = rb = rt = 0
    for h in bottom:
        if h:
            c = TYPE[h]
            b += h.bit_count() * W[c]
            rb += c == 866
    for h in top:
        if h:
            c = TYPE[h]
            t += h.bit_count() * W[c]
            rt += c == 41
    m = d.bit_count()
    a = sum((d & NEIGH[6][i]).bit_count() for i in bits(d))
    s = K*m + b-t - (D//5)*a
    return m, b, t, rb, rt, s

for d in SPECIAL:
    assert canonical6(d) == d
    m, b, t, rb, rt, s = stats6(d)
    if d in TYPES_T:
        assert s == 0 and rb == rt
EXKEY = {}
for d in SPECIAL:
    EXKEY.setdefault(stats6(d), set()).add(d)
assert len(EXKEY) == 13

seen_exceptional = set()
def split6(d):
    if not d:
        return 0, 0
    key = stats6(d)
    m, b, t, rb, rt, s = key
    p = 2*E*(rt-rb)
    if key in EXKEY or s < E*(m+4*rb+4*rt):
        c = canonical6(d)
```

```
        assert c in SPECIAL, ('unlisted six-type', c)
        assert c in EXKEY.get(key, set())
        seen_exceptional.add(c)
        if c in TYPES_S:
            p = SPECIAL[c]
        else:
            assert s == 0 and rb == rt
    return s+2*p, s-2*p

OLD6 = [IX[5][norm(tuple(a for a in p if a != 5))] for p in P[6]]
new6 = [g for new, g in GROUPS[6] if new]
six_total = 0
for k, c in enumerate(reps):
    lift = sum(1 << i for i, old in enumerate(OLD6) if (c >> old) & 1)
    gg = [{lift & m for m in g} - {0} for g in new6]
    front = refine({lift}, sorted(gg, key=len))
    six_total += len(front)
    for h in front:
        split6(h)
    if (k+1) % 20 == 0 or k+1 == len(reps):
        print('Six-label seeds:', k+1, '/', len(reps),
              'extensions:', six_total, 'seconds:',
              round(monotonic()-start, 1), flush=True)
assert seen_exceptional == set(SPECIAL)

# The unique eight-order T-type: its two receiver counts vanish.
R8 = next(d for d in TYPES_T if d.bit_count() == 8)
assert sum(d.bit_count() == 8 for d in TYPES_T) == 1
labels8 = '524130 524310 534120 534210 542130 542310 543120 543210'
given8 = sum(1 << IX[6][tuple(map(int, p))] for p in labels8.split())
assert canonical6(given8) == R8
m, b, t, rb, rt, s = stats6(R8)
assert rb == rt == s == 0
orders = [P[6][i] for i in bits(R8)]
assert len({p[0] for p in orders}) == len({p[-1] for p in orders}) == 1
assert {TYPE[h] for h in slices(6, R8)[0] if h} == {41}
assert {TYPE[h] for h in slices(6, R8)[1] if h} == {866}
split_cached = cache(split6)

def all_meet(ms, full):
    for m in ms:
        full &= m
    return full

def extensions7(seed, endpoint):
    inds = [i for i, p in enumerate(P[7]) if p[endpoint] == 6]
    block = sum(1 << i for i in inds)
    req = sum(1 << i for i in inds
              if (seed >> IX[6][tuple(a for a in P[7][i] if a != 6)]) & 1)
    gg = [[m for m in g if (req & m) == req]
          for _, g in sorted(GROUPS[7], key=lambda g: g[0])]
    assert all(gg)
    base = FULL[7]
    while True:
        old = base
        for g in gg:
            v = {base & m for m in g}
            if len(v) == 1:
                base = next(iter(v))
        if base == old:
            break
    gg = [{base & m for m in g} for g in gg]
    gg = [g for g in gg if len(g) > 1]
    suffix = [FULL[7]] * (len(gg)+1)
    for i in range(len(gg)-1, -1, -1):
        suffix[i] = suffix[i+1] & all_meet(gg[i], FULL[7])
    front = {base}
```

```
    for i, g in enumerate(gg):
        bad = block & suffix[i+1] & ~req
        front = {z for d in front for m in g
                 if not ((z := d & m) & bad)}
    return (d for d in front if (d & block) == req)

seven_counts = []
for seed in sorted(set(SPECIAL) - {R8}):
    for endpoint in (-1, 0):
        count = 0
        for j in extensions7(seed, endpoint):
            bottom, top = slices(7, j)
            lhs = sum(split_cached(h)[0] for h in bottom)
            lhs += sum(split_cached(h)[1] for h in top)
            assert lhs >= 2*E*j.bit_count(), ('seven failure', seed, j)
            count += 1
        seven_counts.append(count)
        print('Seven-label seeds:', len(seven_counts), '/ 26',
              'families:', count, 'seconds:',
              round(monotonic()-start, 1), flush=True)
assert len(seven_counts) == 26
result = dict(result='PASS', five_families=len(F5), five_types=len(reps),
              six_extensions=six_total, exceptional_six_types=len(SPECIAL),
              seven_seed_checks=26, seven_counts=seven_counts,
              seven_family_cases=sum(seven_counts),
              seconds=monotonic()-start)
print(json.dumps(result, indent=2))
```